System Architecture Blueprint

Workflow-Driven B2B Trading Platform

Project Branding:	ceylonherbalspices
Document Version:	v1.0.0 (System Blueprint)
Target Stack:	Next.js (React), NestJS (Node.js), RabbitMQ, PostgreSQL
Environment:	Hybrid (Windows Local Host / WSL2 Ubuntu Dev Environment)

1. Executive Architectural Summary

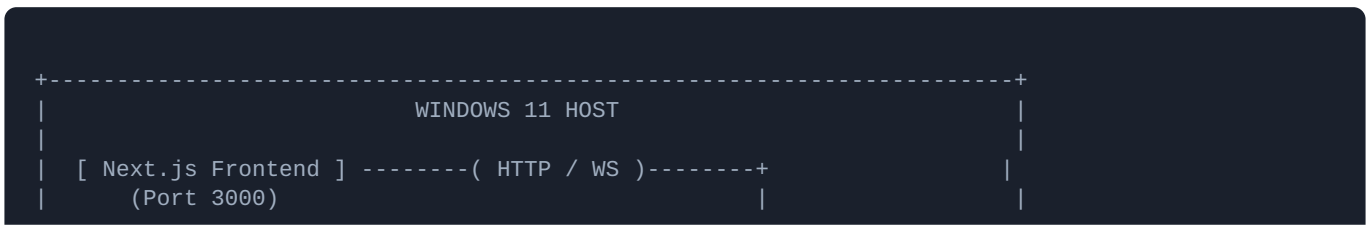
This document outlines the core structural blueprint for the **ceylonherbalspices** B2B trading platform. Unlike consumer-oriented B2C applications, a high-volume B2B platform operates on long-running, multi-party business workflows—such as Bulk RFQ processing, price negotiation, multi-tiered organizational approvals, escrow configurations, and international customs clearance.

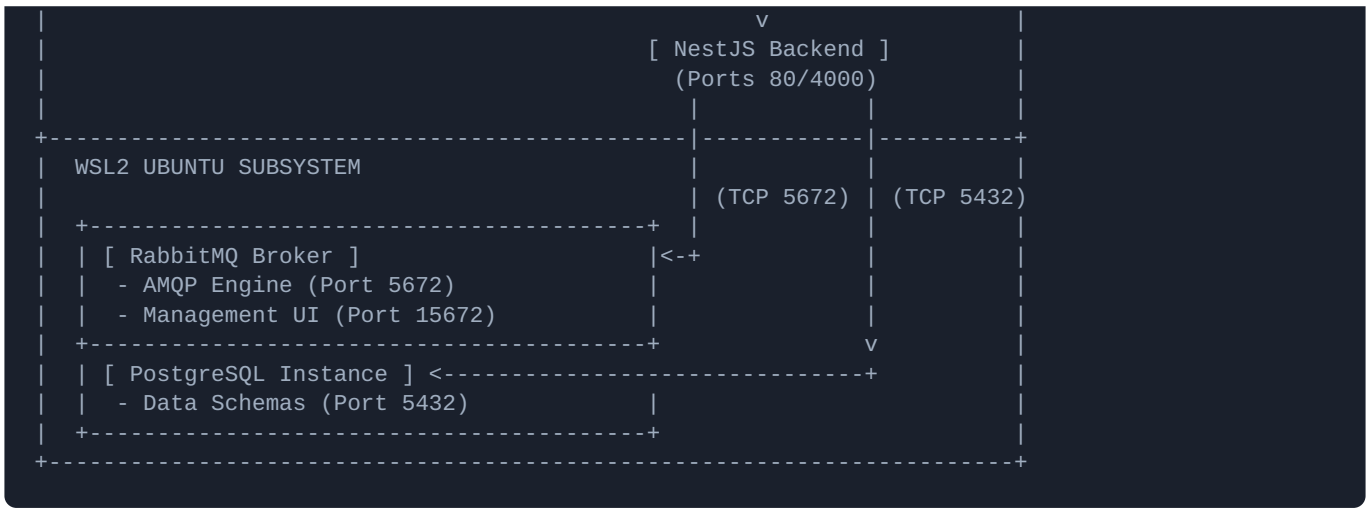
To ensure scalability, decoupled microservices, and fault tolerance across asynchronous boundaries, the platform leverages an event-driven infrastructure. The system decouples incoming synchronous communications via an **API Gateway** and orchestrates all downstream background processing using **RabbitMQ** as the asynchronous core, backed by transactional state synchronization within a **PostgreSQL** operational layer.

2. System Topology & Development Layout

To optimize developer workspace velocity while preserving production environment alignment, the engineering workspace uses a cross-boundary architecture between native Windows 11 and Windows Subsystem for Linux (WSL2 Ubuntu):

- **Client Tier (Windows):** Next.js (React) executing inside native Windows, utilizing server-side rendering (SSR) and reactive UI components.
- **Application Tier (Windows):** Node.js & NestJS framework running natively on Windows, communicating via standard TCP loopback addresses.
- **Infrastructure Tier (WSL2 Ubuntu):** Isolated PostgreSQL database instances and a native RabbitMQ server broker running within Linux systemd processes.





3. The API Gateway: The Intelligent Front Door

The API Gateway acts as the uniform edge boundary for all inbound client traffic and programmatic integrations. It standardizes traffic processing before requests ever hit the backend domain services.

3.1 Core Security & Protocol Directives

- **Authentication & Cryptographic Extraction:** Ingests external JSON Web Tokens (JWT) or corporate API keys. It decodes claims, verifies signatures, and injects context headers (e.g., `X-Tenant-ID`, `X-User-Role`) into downstream requests to enforce corporate logical isolation.
- **Ingress Rate Limiting:** Enforces dynamic traffic throttling. Standard guest users are limited to $R_{guest} = 60$ requests/minute, while registered corporate buyers enjoy elevated service levels ($R_{corp} = 1000$ requests/minute).
- **Payload Normalization:** Translates outbound Webhook formats and incoming ERP data streams into unified JSON contracts processed by NestJS.

3.2 Asynchronous Handoff Strategy

For critical B2B transaction operations, the API Gateway actively prevents blocking connections. Upon receipt of a workflow invocation request (e.g., `POST /api/v1/rfq`), the gateway routes the request to the target domain service, which immediately issues an HTTP status code `202 Accepted` containing an execution trace ID. The thread is liberated to handle subsequent edge requests while the transaction shifts entirely to the asynchronous bus.

4. The Event Bus: The Asynchronous Nervous System

The messaging framework relies entirely on RabbitMQ running within WSL2 Ubuntu. This tier decouples services, orchestrates multi-step state transitions, and guarantees eventual consistency across backend databases.

4.1 Advanced Routing Topology

To support complex workflow routing, the system utilizes RabbitMQ **Topic Exchanges** (e.g., `amq.topic`). This approach allows microservices to subscribe to precise subsets of business events using dot-notation routing keys (e.g., `order.srilanka.cinnamon.placed`).

> **Core Structural Configurations:**
>
> - **Durable Exchanges & Queues:** All exchanges and queues are instantiated with `durable: true`. This guarantees that if the WSL2 subsystem restarts, the structural layout and unprocessed B2B transactions are safely recovered from storage.
> - **Message Persistence:** Messages are dispatched with delivery mode set to 2 (Persistent), forcing RabbitMQ to flush them safely to disk.

4.2 Connection Configuration Example

Due to WSL2's native automated networking bridge, the NestJS application running on the Windows host safely connects to RabbitMQ via standard loopback strings:

```
// nestjs-rabbitmq-config.ts
import { NestFactory } from '@nestjs/core';
import { MicroserviceOptions, Transport } from '@nestjs/microservices';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.createMicroservice<MicroserviceOptions>(AppModule, {
    transport: Transport.RMQ,
    options: {
      urls: ['amqp://localhost:5672'], // Loops securely into WSL2
      queue: 'ceylonherbalspices_workflow_queue',
      queueOptions: {
        durable: true,
      },
      noAck: false, // Forces manual message acknowledgment
    },
  });
  await app.listen();
}
bootstrap();
```

5. End-to-End Execution Lifecyle: Bulk B2B Order Workflow

To visualize how the API Gateway and RabbitMQ interact, the following execution lifecycle tracks a high-value bulk spice transaction where an international buyer requests an export allotment exceeding $10,000, triggering an automatic multi-party approval sequence.

Step	Component	Communication Style	Architectural Action Description
1	Next.js Client	Synchronous HTTPS	Buyer clicks "Submit Bulk Purchase Request" for 5 Metric Tons of Black Pepper. Dispatches payload to Windows Host.
2	API Gateway	Synchronous Internal	Validates token, confirms `ceylonherbalspices` tenancy space, and passes request payload to Order Service.
3	Order Service	Internal Write	Commits a database record into PostgreSQL with state set to `AWAITING_APPROVAL`. Immediately releases connection.
4	Order Service	Asynchronous Publish	Dispatches message to RabbitMQ topic exchange with routing key `order.bulk.submitted`. Returns `202 Accepted` to frontend.
5	Workflow Engine	Asynchronous Consume	Pulls message from RabbitMQ. Evaluates business rules. Identifies dollar threshold > $10,000; locks workflow state machine.
6	Notification Service	Asynchronous Fan-out	Consumes identical event parallelly; triggers a WebSocket push notification to the administrative manager's dashboard.

6. Key Cross-Cutting Concerns for This Architecture

Operating distributed workflows across asynchronous components requires strict mechanisms to avoid data corruption, duplicate execution, and messaging deadlocks.

6.1 Idempotency Mechanisms

Network packet retries mean NestJS consumers can receive identical events multiple times. To safeguard against duplicated processing, every action generates a unique deterministic `Idempotency-Key` at the API Gateway level.

Before a NestJS consumer executes a transaction block, it checks for the existence of this key inside PostgreSQL using an atomic operation:

```
// Pseudo-logic for idempotent execution block
INSERT INTO processed_events (event_id, processed_at)
VALUES ('evt_order_998231', NOW())
ON CONFLICT (event_id) DO NOTHING;
```

If the query returns zero rows affected, the message is safely discarded as a duplicate, protecting monetary systems from accidental double processing.

6.2 The Transactional Outbox Pattern

To prevent scenarios where a database update completes but a network partition blocks the subsequent RabbitMQ event dispatch, the system strictly enforces the **Transactional Outbox Pattern**.

Instead of pushing directly to RabbitMQ during an active HTTP block, the NestJS application writes the business record and an Event log into the same PostgreSQL database inside an atomic transaction block:

```
BEGIN TRANSACTION;
  UPDATE orders SET status = 'APPROVED' WHERE id = 104;
  INSERT INTO outbox_table (id, aggregate_type, payload, status)
  VALUES ('msg_01', 'ORDER', '{"id": 104, "status": "APPROVED"}', 'PENDING');
COMMIT;
```

An independent background polling daemon running within NestJS reads from the `outbox_table`, dispatches the event safely to RabbitMQ, and marks the status as `PROCESSED` only after receiving a positive acknowledgment from the broker.

6.3 Outbound Enterprise Webhooks

International enterprise buyers often require automated system-to-system data syncing back to their custom ERP platforms. The event bus addresses this via a dedicated **Webhook Outbound Dispatcher Service**.

This microservice consumes events like `shipment.customs_cleared` from RabbitMQ, queries user routing preferences, and wraps the event in an authorized outbound HTTPS POST request directly to the client's destination endpoints, executing robust retry patterns using exponential backoff schedules.